

Лабораторная работа № 3. Управляющие структуры.

Абакумова О. М.

Российский университет дружбы народов, Москва, Россия

Информация

- Абакумова Олеся Максимовна
- Студентка
- Российский университет дружбы народов
- 1132220832@pfur.ru
- <https://github.com/omabakumova>



Цель работы

Цель работы

Основная цель работы — освоить применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

Выполнение лабораторной работы

Циклы while и for

Циклы while и for

```
1) # цикл while приближает n к единице и рассчитывает логарифм:
n = 10
while n > 1
  n = n / 2
  print(n)
end

1
0.5
0.25
0.125
0.0625
0.03125
0.015625
0.0078125
0.00390625
0.001953125

2) myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
i = 1
while i <= length(myfriends)
  friend = myfriends[i]
  print("Hi friend, it's great to see you!")
  i = i + 1
end

Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!

3) for n in 1:2:10
  print(n)
end

1
3
5
7
9

4) myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
for friend in myfriends
  print("Hi friend, it's great to see you!")
end

Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!

5) # генерируем матрицу n x n из нулей
n = 5
A = fill{0, (n, n)}

6) 5x5 Matrix{Int64}:
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Рис. 1: Примеры с while и for

Циклы while и for

```
1: # двумерный массив, в котором значение каждой ячейки  
# является суммой индексов строки и столбца  
for i in 1:n  
    for j in 1:n  
        A[i, j] = i + j  
    end  
end  
A  
5x5 Matrix{Int64}:  
2 3 4 5 6  
3 4 5 6 7  
4 5 6 7 8  
5 6 7 8 9  
6 7 8 9 10  
1: # инициализация массива и n = 0 нулей:  
B = Fill{0, 5n, 0}  
5x5 Matrix{Int64}:  
0 0 0 0 0  
0 0 0 0 0  
0 0 0 0 0  
0 0 0 0 0  
0 0 0 0 0  
1: for i in 1:n, j in 1:n  
    B[i, j] = i + j  
end  
B  
5x5 Matrix{Int64}:  
2 3 4 5 6  
3 4 5 6 7  
4 5 6 7 8  
5 6 7 8 9  
6 7 8 9 10  
1: C = 1 + j for i in 1:n, j in 1:n  
C  
5x5 Matrix{Int64}:  
2 3 4 5 6  
3 4 5 6 7  
4 5 6 7 8  
5 6 7 8 9  
6 7 8 9 10
```

Рис. 2: Использование цикла for для создание двумерного массива

Довольно часто при решении задач требуется проверить выполнение тех или иных условий. Для этого используют условные выражения.

Условные выражения

```
- Условные выражения
1: N = 15
2: 15
3:
4: # используем 'is' для проверки операции "AND"
5: # проверяем % вычисляет остаток от деления
6: if (N % 3 == 0) and (N % 5 == 0):
7:     print("FizzBuzz")
8: elif N % 3 == 0:
9:     print("Fizz")
10: elif N % 5 == 0:
11:     print("Buzz")
12: else:
13:     print(N)
14: end
15: FizzBuzz
16:
17: x = 5
18: 5
19:
20: y = 10
21: 10
22:
23: (x > y) ? x : y
24: 10
```

Рис. 3: Пример условного выражения

```
Функции
1301 function sayhi(name)
1302     println("Hi $name, it's great to see you!")
1303 end
1304 sayhi (generic function with 1 method)
1305
1306 # dynamic anonymous n-arguer:
1307 function f(x)
1308     x^2
1309 end
1310 f (generic function with 1 method)
1311 sayhi("C. 309")
1312     Hi C-309, it's great to see you!
1313 f(42)
1314     1764
1315 sayhi2(name) = println("Hi $name, it's great to see you!")
1316 sayhi2 (generic function with 1 method)
1317 f2(x) = x^2
1318 f2 (generic function with 1 method)
1319 sayhi3 = name -> println("Hi $name, it's great to see you!")
1320 #3 (generic function with 1 method)
1321 f2 = x -> x^2
1322 #5 (generic function with 1 method)
1323
1324 # simple vector v:
1325 v = [1, 5, 2]
1326 part(v)
1327     3-element Vector{Int64}:
1328      1
1329      5
1330      2
1331 part(v)
1332     3-element Vector{Int64}:
1333      1
1334      5
1335      2
1336 map(f, [1, 2, 3])
1337     3-element Vector{Int64}:
1338      1
1339      4
1340      9
```

Рис. 4: Пример написания функции

По соглашению в Julia функции, сопровождаемые восклицательным знаком, изменяют свое содержимое, а функции без восклицательного знака не делают этого

Функция `sort(v)` возвращает отсортированный массив, который содержит те же элементы, что и массив `v`, но исходный массив `v` остаётся без изменений. Если же использовать `sort!(v)`, то отсортировано будет содержимое исходного массива `v`.

В программировании под функцией высшего порядка понимается функция, принимающая в качестве аргументов другие функции или возвращающая другую функцию в качестве результата. Основная идея состоит в том, что функции имеют тот же статус, что и другие объекты данных.

```
1421) f(x) = x^2
      map(f, [1, 2, 3])

1422) 3-element Vector{Int64}:
      1
      4
      9

1423) x -> x^3
      map(x -> x^3, [1, 2, 3])

1424) 3-element Vector{Int64}:
      1
      8
      27

1425) f(x) = x^2
      broadcast(f, [1, 2, 3])

1426) 3-element Vector{Int64}:
      1
      4
      9

1427) f = [1, 2, 3]

1428) 3-element Vector{Int64}:
      1
      2
      3

1429) # Zigzag sequence A:
      A = [1 + 2^j for j in 0:2, 1 in 1:3]

1430) 3x3 Matrix{Int64}:
      1  2  3
      4  5  6
      7  8  9

1431) f(A)

1432) 3x3 Matrix{Int64}:
      10  30  40
      40  80  90
      100 120 130

1433) B = f.(A)

1434) 3x3 Matrix{Int64}:
      1  4  9
      16 25 36
      49 64 81

1435) A -> 2 * f.(A) / A

1436) 3x3 Matrix{Float64}:
      2.0  4.0  6.0
      12.0 15.0 18.0
      21.0 24.0 27.0

1437) @. A = 2 * f(A) / A

1438) 3x3 Matrix{Float64}:
      2.0  4.0  6.0
      12.0 15.0 18.0
      21.0 24.0 27.0

1439) broadcast(x -> x + 2 * f(x) / x, A)

1440) 3x3 Matrix{Float64}:
      3.0  6.0  9.0
      14.0 18.0 21.0
      23.0 28.0 31.0
```

Рис. 5: Использование функций

- В Julia функция `map` является функцией высшего порядка, которая принимает функцию в качестве одного из своих входных аргументов и применяет эту функцию к каждому элементу структуры данных, которая ей передаётся также в качестве аргумента.

- Функция **broadcast** — ещё одна функция высшего порядка в Julia, представляющая собой обобщение функции **map**. Функция **broadcast()** будет пытаться привести все объекты к общему измерению, **map()** будет напрямую применять данную функцию поэлементно.

- Точечный синтаксис для `broadcast()` позволяет записать относительно сложные составные поэлементные выражения в форме, близкой к математической записи.

Сторонние библиотеки (пакеты) в Julia

Julia имеет более 2000 зарегистрированных пакетов, что делает их огромной частью экосистемы Julia. Есть вызовы функций первого класса для других языков, обеспечивающие интерфейсы сторонних функций. Можно вызвать функции из Python или R, например, с помощью `PyCall` или `Rcall`.

Сторонние библиотеки (пакеты) в Julia

```
Сторонние библиотеки (пакеты) в Julia

julia> import Plg

julia> Plg.add("Example")

Updating registry at ~/.julia/registries/General.toml
Resolving package versions...
Installed Example = v0.3.5
Updating ~/.julia/environments/v1.11/Project.toml
[7fba0ff] + Example v0.3.5
Updating ~/.julia/environments/v1.11/Manifest.toml
[7fba0ff] + Example v0.3.5
Precompiling project...
  50d/6 m - / Example
  1 dependency successfully precompiled in 10 seconds. 850 already precompiled.

julia> using Example

julia> Plg.add("Colors")

Resolving package versions...
No changes to ~/.julia/environments/v1.11/Project.toml
No changes to ~/.julia/environments/v1.11/Manifest.toml

julia> using Colors

julia> palette = distinguishable_colors(100)

julia>

julia> rand(palette, 2, 2)
```

Рис. 6: Установка пакета, его подключение и пример с его использованием

Задания для самостоятельной работы

1. Используя циклы `while` и `for`:

- выведите на экран целые числа от 1 до 100 и напечатайте их квадраты;
- создайте словарь `squares`, который будет содержать целые числа в качестве ключей и квадраты в качестве их пар-значений;
- создайте массив `squares_arr`, содержащий квадраты всех чисел от 1 до 100.

Задания для самостоятельной работы

```
Задания для самостоятельного выполнения
1) using System;

# 2. Даны искомые
println("1. Число и его квадраты")
# while
1. i = 1
while i <= 100
    println(i + " -> " + i*i)
    i = i + 1
end

1. Число и его квадраты:
1 -> 1
2 -> 4
3 -> 9
4 -> 16
5 -> 25
6 -> 36
7 -> 49
8 -> 64
9 -> 81
10 -> 100
11 -> 121
12 -> 144
13 -> 169
14 -> 196
15 -> 225
16 -> 256
17 -> 289
18 -> 324
19 -> 361
20 -> 400
21 -> 441
22 -> 484
23 -> 529
24 -> 576
25 -> 625
26 -> 676
27 -> 729
28 -> 784
29 -> 841
30 -> 900
31 -> 961
32 -> 1024
33 -> 1089
34 -> 1156
35 -> 1225
36 -> 1296
37 -> 1369
38 -> 1444
39 -> 1521
40 -> 1600
41 -> 1681
42 -> 1764
```

Рис. 7: Задание 1

Задания для самостоятельной работы

```
11881: # for
      for i in 1:200
        println("xi ->  $x(i*2)^2$ ")
      end

1 -> 1
2 -> 4
3 -> 9
4 -> 16
5 -> 25
6 -> 36
7 -> 49
8 -> 64
9 -> 81
10 -> 100
11 -> 121
12 -> 144
13 -> 169
14 -> 196
15 -> 225
16 -> 256
17 -> 289
18 -> 324
19 -> 361
20 -> 400
21 -> 441
22 -> 484
23 -> 529
24 -> 576
25 -> 625
26 -> 676
27 -> 729
28 -> 784
29 -> 841
30 -> 900
31 -> 961
32 -> 1024
33 -> 1089
34 -> 1156
35 -> 1225
36 -> 1296
37 -> 1369
38 -> 1444
39 -> 1521
40 -> 1600
41 -> 1681
42 -> 1764
43 -> 1849
44 -> 1936
45 -> 2025
46 -> 2116
47 -> 2209
48 -> 2304
49 -> 2401
50 -> 2500
51 -> 2601
52 -> 2704
```

Рис. 8: Задание 1

Задания для самостоятельной работы

```
1: # Скорость
squares = Dict{Int, Int}()
for i in 1:100
    squares[i] = i^2
end

1: # Массив
squares_arr = [i^2 for i in 1:100]

1: 100-element Vector{Int64}:
 1
 4
 9
16
25
36
49
64
81
100
121
144
169
196
225
256
289
324
361
400
441
484
529
576
625
676
729
784
841
900
961
1024
1089
1156
1225
1296
1369
1444
1521
1600
1681
1764
1849
1936
2025
2116
2209
2304
2401
2500
2601
2704
2809
2916
3025
3136
3249
3364
3481
3600
3721
3844
3969
4096
4225
4356
4489
4624
4761
4900
5041
5184
5329
5476
5625
5776
5929
6084
6241
6400
6561
6724
6889
7056
7225
7396
7569
7744
7921
8100
8281
8464
8649
8836
9025
9216
9409
9604
9801
10000

1: # 2. Проверка чётности
function check_parity(n)
    if n % 2 == 0
        println(n)
    else
        println("oddness")
    end
    check_parity(11)
end

1: # Трёхный оператор
function check_parity_ternary(n)
    println(n, 2 == 0 ? "evenness" : "oddness")
end
check_parity_ternary(3)

1: # 2. Remove add_one
add_one(x) = x + 1
add_one(x)

1: 1
```

Рис. 9: Задание 1, 2 и 3

2. Напишите условный оператор, который печатает число, если число чётное, и строку «нечётное», если число нечётное. Перепишите код, используя тернарный оператор.

Задания для самостоятельной работы

```
1: # Скорость
squares = Dict{Int, Int}()
for i in 1:100
    squares[i] = i^2
end

1: # Массив
squares_arr = [i^2 for i in 1:100]

1: 100-element Vector{Int64}:
 1
 4
 9
16
25
36
49
64
81
100
121
144
169
196
225
256
289
324
361
400
441
484
529
576
625
676
729
784
841
900
961
1024
1089
1156
1225
1296
1369
1444
1521
1600
1681
1764
1849
1936
2025
2116
2209
2304
2401
2500
2601
2704
2809
2916
3025
3136
3249
3364
3481
3600
3721
3844
3969
4096
4225
4356
4489
4624
4761
4900
5041
5184
5329
5476
5625
5776
5929
6084
6241
6400
6561
6724
6889
7056
7225
7396
7569
7744
7921
8100
8281
8464
8649
8836
9025
9216
9409
9604
9801
10000

1: # 2. Проверка чётности
function check_parity(n)
    if n % 2 == 0
        println(n)
    else
        println("odd")
    end
    check_parity(11)
end

1: # Трёхный оператор
function check_parity_ternary(n)
    println(n, 2 == 0 ? "even" : "odd")
end
check_parity_ternary(3)

1: x = 2
# 2. Remove add_one
add_one(x) = x + 1
add_one(x)

1: 3
```

Рис. 10: Задание 1, 2 и 3

3. Напишите функцию `add_one`, которая добавляет 1 к своему входу.

Задания для самостоятельной работы

```
1: # Скорость
squares = Dict{Int, Int}()
for i in 1:100
    squares[i] = i^2
end

1: # Массив
squares_arr = [i^2 for i in 1:100]

1: 100-element Vector{Int64}:
 1
 4
 9
16
25
36
49
64
81
100
121
144
169
196
225
256
289
324
361
396
441
484
529
576
625
676
729
784
841
900
961
1024
1089
1156
1225
1296
1369
1444
1521
1600
1681
1764
1849
1936
2025
2116
2209
2304
2401
2500
2601
2704
2809
2916
3025
3136
3249
3364
3481
3600
3721
3844
3969
4096
4225
4356
4489
4624
4761
4900
5041
5184
5329
5476
5625
5776
5929
6084
6241
6400
6561
6724
6889
7056
7225
7396
7569
7744
7921
8100
8281
8464
8649
8836
9025
9216
9409
9604
9801
10000

1: # 2. Проверка чётности
function check_parity(n)
    if n % 2 == 0
        println(n)
    else
        println("oddness")
    end
    check_parity(11)
end

1: oddness

1: # Третий оператор
function check_parity_hermey(n)
    println(n % 2 == 0 ? n : "oddness")
end
check_parity_hermey(3)

1: oddness

1: x = 2
# 2. Remove add_one
add_one(x) = x + 1
add_one(x)

1: 3
```

Рис. 11: Задание 1, 2 и 3

- Используйте `map()` или `broadcast()` для задания матрицы A, каждый элемент которой увеличивается на единицу по сравнению с предыдущим.

Задания для самостоятельной работы

```
12) # 4. Матрица с независимыми значениями
A = reshape(1:9, 3, 3) %>% Matrix
A_plus_one = broadcast(x = 1, A)
```

```
13) 3x3 Matrix (int64):
 2  5  8
 3  6  9
 4  7 10
```

```
13) # 5. Задать с помощью A
A = [1 1 3; 5 2 6; -2 -1 -3]
k(x) = x^2
B = k(A)
```

```
13) 3x3 Matrix (int64):
 1  1  29
125  6 216
 8 -1 -27
```

```
14) # Заполнить матрицу строками
for i in 1:3
  A[i,i] = A[i,1] + A[i,2]
end
A
```

```
14) 3x3 Matrix (int64):
 1  1  2
 5  2  7
-2 -1 -3
```

```
15) m, n = 2, 3
A = fill(1, (m, n))
```

```
15) 2x3 Matrix (int64):
 1  1  1
 1  1  1
```

```
17) # 6. Матрица B в произведении
B = [10 -10 10]
B = repeat(B, 15)
C = B * B
```

```
17) 3x3 Matrix (int64):
1500 -1500 1500
1500 1500 -1500
1500 -1500 1500
```

Рис. 12: Задание 4, 5 и 6

5. Задайте матрицу A и выполните:

- Возведите матрицу в третью степень.
- Замените третий столбец матрицы A на сумму второго и третьего столбцов.

Задания для самостоятельной работы

```
121 # 4. Перенос с несимметричного индексируемого  
A = reshape(1:9, 3, 3) %>% Matrix  
A_plus_one = broadcast(x = x + 1, A)
```

```
141 3x3 Matrix [int64]:  
 2  5  8  
 3  6  9  
 4  7 10
```

```
121 # 5. Задать с помощью A  
A = [1 1 3; 5 2 6; -2 -1 -3]  
k(x) = x^2  
B = k(A)
```

```
131 3x3 Matrix [int64]:  
 1  1  27  
125  8 216  
 8 -1 -27
```

```
141 # Замена первого столбца  
for i in 1:3  
  A[i,1] = A[i,1] + A[i,2]  
end  
A
```

```
141 3x3 Matrix [int64]:  
 1  1  2  
 5  2  7  
 -2 -1 -3
```

```
151 m, n = 2, 3  
A = fill(1, (m, n))
```

```
151 2x3 Matrix [int64]:  
 1  1  1  
 1  1  1
```

```
171 # 6. Матрица B в произведении  
B = [10 -10 10]  
B = repeat(B, 15)  
C = B * B
```

```
171 3x3 Matrix [int64]:  
1500 -1500 1500  
1500 1500 -1500  
1500 -1500 1500
```

Рис. 13: Задание 4, 5 и 6

6. Создайте матрицу B с элементами. Вычислите матрицу $C = B^T B$.

Задания для самостоятельной работы

```
121: # 4. Матрица с независимыми значениями
A = reshape(1:9, 3, 3) %>% Matrix
A_plus_one = broadcast(x = x + 1, A)

141: 3x3 Matrix[Integer64]:
  2  5  8
  3  6  9
  4  7 10

161: # 5. Задать с помощью A
A = [1 1 3; 5 2 6; -2 -1 -3]
k(x) = x^2
B = k(A)

181: 3x3 Matrix[Integer64]:
  1  1  27
125  6 216
  8 -1 -27

201: # Замена первого столбца
for i in 1:3
  A[i,1] = A[i,1] + A[i,2]
end
A

241: 3x3 Matrix[Integer64]:
  1  1  2
  5  2  7
  2 -1 -3

261: m, n = 2, 3
A = fill(1, (m, n))

281: 2x3 Matrix[Integer64]:
  1  1  1
  1  1  1

301: # 6. Матрица B в произведении
B = [10 -10 10]
B = repeat(B, 15)
C = B * B

321: 3x3 Matrix[Integer64]:
1500 -1500 1500
1500 1500 -1500
1500 -1500 1500
```

Рис. 14: Задание 4, 5 и 6

7. Создайте матрицу Z размерности 6×6 , все элементы которой равны нулю, и матрицу E , все элементы которой равны 1. Используя циклы **while** или **for** и закономерности расположения элементов, создайте следующие матрицы размерности 6×6 .

Задания для самостоятельной работы

```
76. # Задание 7 {  
# Создаем нулевую матрицу 6x6 и матрицу из единиц 6x6  
Z = fill(0, (6, 6))  
  
161: 6x6 Matrix{Int64}:  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 0 0 0  
  
171: E = fill(1, (6, 6))  
  
171: 6x6 Matrix{Int64}:  
1 1 1 1 1 1  
1 1 1 1 1 1  
1 1 1 1 1 1  
1 1 1 1 1 1  
1 1 1 1 1 1  
1 1 1 1 1 1  
  
198. # Создаем нулевые матрицы  
Z1 = zeros{Int, 6, 6}  
Z2 = zeros{Int, 6, 6}  
Z3 = zeros{Int, 6, 6}  
Z4 = zeros{Int, 6, 6}  
  
161: 6x6 Matrix{Int64}:  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 0 0 0  
0 0 0 0 0 0  
  
121: # Z1 - единицы на побочных диагоналях ([i-j] == 1)  
for i in 1:6, j in 1:6  
    if abs(i - j) == 1  
        Z1[i, j] = 1  
    end  
end  
println("Z1:")  
display(Z1)  
  
Z1:  
6x6 Matrix{Int64}:  
0 1 0 0 0 0  
1 0 1 0 0 0  
0 1 0 1 0 0  
0 0 1 0 1 0  
0 0 0 1 0 1  
0 0 0 0 1 0  
  
131: # Z2 - единицы на главной диагонали и через одну позиции ([i-j] == 0 или 2)  
for i in 1:6, j in 1:6  
    if i == j || abs(i - j) == 2  
        Z2[i, j] = 1  
    end  
end  
println("\nZ2:")  
display(Z2)  
  
Z2:  
6x6 Matrix{Int64}:  
1 0 1 0 0 0  
0 1 0 1 0 0  
1 0 1 0 1 0  
0 1 0 1 0 1  
0 0 1 0 1 0  
0 0 0 1 0 1
```

Рис. 15: Задание 7

Задания для самостоятельной работы

```
141: # Z3 - единицы в изоматрице порядка n с определенным паттерном
# Альтернативный подход с использованием индексов
Z3[1, [4, 6]] := 1
Z3[2, [3, 5]] := 1
Z3[3, [2, 4, 6]] := 1
Z3[4, [1, 3, 5]] := 1
Z3[5, [2, 4]] := 1
Z3[6, [1, 3]] := 1
println("Z3:")
display(Z3)

Z3:
0x0 Matrix{Int64}:
0 0 1 0 1
0 1 0 1 0
0 1 0 1 0
1 0 1 0 1
1 0 1 0 1
0 1 0 1 0
1 0 1 0 0
1 0 1 0 0

151: # Z4 - единицы на четных суммах индексов ((i+j) % 2 == 0)
for i in 1:6, j in 1:6
    if (i + j) % 2 == 0
        Z4[i, j] = 1
    end
end
println("Z4:")
display(Z4)

Z4:
0x0 Matrix{Int64}:
1 0 1 0 1
1 0 1 0 1
0 1 0 1 0
0 1 0 1 0
1 0 1 0 1
0 1 0 1 0
```

Рис. 16: Задание 7

8. В языке R есть функция `outer()`. Фактически, это матричное умножение с возможностью изменить применяемую операцию (например, заменить произведение на сложение или возведение в степень)/ Напишите эту функцию.

Задания для самостоятельной работы

```
x x x x x
221: # Задание 8

# Функция outer выполняет операции между каждым элементом вектора x и y
# x ~ "y" - возведение в степень (полное)
# x ~ "y" - вычитание (полное)
# x ~ "y" - взятие по модулю (полное)
function outer(x, y, f)
  if (f == "+")
    res = x ~ "y" # Возведение в степень
  elseif (f == "-")
    res = x - y
  elseif (f == "**")
    res = x ~ "y" # Возведение в степень (дублирование условия)
  elseif (f == "%")
    res = x % y
  else
    res = x ~ "y" # Вычитание
  end
  res = x ~ "y" # Взятие по модулю
end

222: outer (generic function with 1 method)

223: # Создание матрицы SxS с операциями "+" 7777777777777777
A1 = outer(0:4, 0:4, "+")
A1

224: SxS Matrix{Int64}:
 0 1 2 3 4
 1 2 3 4 5
 2 3 4 5 6
 3 4 5 6 7
 4 5 6 7 8

225: # Создание матрицы SxS с операциями "-"
A2 = outer(0:4, 1:5, "-")

226: SxS Matrix{Int64}:
 0 8 9 0 0
 1 1 1 1 1
 2 4 8 16 32
 3 9 27 81 243
 4 16 64 256 1024

227: A3 = outer(0:4, 0:4, "+")
A3_res = outer(A3, 5, "%")

228: SxS Matrix{Int64}:
 0 1 2 3 4
 1 2 3 4 5
 2 3 4 0 1
 3 4 0 1 2
 4 0 1 2 3

229: A3 = outer(0:9, 0:9, "+")
A3_res = outer(A3, 10, "%")

230: 10x10 Matrix{Int64}:
 0 1 2 3 4 5 6 7 8 9
 1 2 3 4 5 6 7 8 9 0
 2 3 4 5 6 7 8 9 0 1
 3 4 5 6 7 8 9 0 1 2
 4 5 6 7 8 9 0 1 2 3
 5 6 7 8 9 0 1 2 3 4
 6 7 8 9 0 1 2 3 4 5
 7 8 9 0 1 2 3 4 5 6
 8 9 0 1 2 3 4 5 6 7
 9 0 1 2 3 4 5 6 7 8

231: A3 = outer(0:9, 0:9, "+")
A3 = abs.(A3) # Близко абсолютные значения

232: 10x10 Matrix{Int64}:
 0 1 2 3 4 5 6 7 8 9
 1 0 1 2 3 4 5 6 7 8
 2 1 0 1 2 3 4 5 6 7
 3 2 1 0 1 2 3 4 5 6
 4 3 2 1 0 1 2 3 4 5
 5 4 3 2 1 0 1 2 3 4
 6 5 4 3 2 1 0 1 2 3
 7 6 5 4 3 2 1 0 1 2
 8 7 6 5 4 3 2 1 0 1
 9 8 7 6 5 4 3 2 1 0
```


9. Решите следующую систему линейных уравнений с 5 неизвестными:

$$\begin{cases} x_1 + 2x_2 + 3x_3 + 4x_4 + 5x_5 = 7, \\ 2x_1 + x_2 + 2x_3 + 3x_4 + 4x_5 = -1, \\ 3x_1 + 2x_2 + x_3 + 2x_4 + 3x_5 = -3, \\ 4x_1 + 3x_2 + 2x_3 + x_4 + 2x_5 = 5, \\ 5x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 = 17, \end{cases}$$

Задания для самостоятельной работы

```
# Задание 9
# Определим матрицу коэффициентов A
A = [
    1 2 3 4 5;
    2 1 2 3 4;
    3 2 1 2 3;
    4 3 2 1 2;
    5 4 3 2 1
]

# Определим вектор правой части y
y = [7; -1; -3; 5; 17]

# Решим систему линейных уравнений A*x = y
x = A \ y

# Выведем решение системы
println("Решение системы: ")
println(x)

Решение системы:
[-2.00000000000000036, 3.00000000000000050, 4.9999999999999998, 1.9999999999999991, -3.9999999999999999]
```

Рис. 18: Задание 9

10. Создайте матрицу M размерности 6×10 , элементами которой являются целые числа.

11. Вычислите следующие выражения.

Задания для самостоятельной работы

```
# Задание 10

# Создаем случайную матрицу 6x10 с числами от 1 до 10
M = randi(10, 6, 10)
println(M)

N = 4
K = 75

# Считаем сумму всех элементов матрицы
count_1 = sum(M, :0)
println(count_1)

# Нам строки, где сумма элементов равна 7
count_2 = [1 for i=1:N if sum(M[i,:]) == 7]
println(count_2)

# Нам две столбца, суммы которых больше K
count_3 = [1, 2] for i = 1:N, j = 2:5 if (M[i, j] + M[i, j+1]) > K]
println(count_3)

[7 6 10 3 10 5 5 3 8; 7 1 5 7 6 4 3 2 7 8; 10 4 4 1 2 2 7 3 3; 8 8 9 10 1 3 10 6 0 1; 7 5 8 7 5 5 2 9 6 4; 1 10 10 6 2 1 5 7 10 10]
38
[5]
[13, 2], (1, 3), (2, 3), (4, 3), (1, 4), (3, 4)]

# Задание 11

# Вычисляем первую двойную сумму
# Для i от 1 до 20 и j от 1 до 5
# Выводим: 1*1 / (3 + 1)
sum1 = sum(1.^4 / (3 + 1)) for i in 1:20, j in 1:5

# Вычисляем вторую двойную сумму
# Для i от 1 до 20 и j от 1 до 5
# Выводим: 1*1 / (3 + 1 + 1)
sum2 = sum(1.^4 / (3 + 1 + 1)) for i in 1:20, j in 1:5

# Выводим результаты
println("Первая сумма: sum1")
println("Вторая сумма: sum2")

Первая сумма: 630215.26323333338
Вторая сумма: 89912.02146957133
```

Рис. 19: Задание 10 и 11

Выводы

В результате выполнения данной лабораторной работы я освоила применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.